

TL2 – план внедрения

Per 🌳🌳 ad ★★

Григорий Бутейко vk:@hrissan

СВОЙСТВА TL2

Расширяемость и простые рецепты

- Легко добавлять и убирать поля
- Нет Boxed/Bare представлений, magic – просто аннотация
- Легко расширить простой тип до Union
- Легко расширить ответ функции
- Bool, optional – продуманный механизм
- Маски Полей – деталь реализации, недоступная пользователю
- Ответы функций читаются и пишутся независимо от полей функций
- Медленнее TL, но гораздо быстрее конкурентов

СВОЙСТВА TL2

Краткий справочник

```
int32 = int; // typedef
```

```
mc.acl = id:long roles:[]int; // object and vector
```

```
mc.response = Empty | WithCode code:int; // union (enum)
```

```
mc.increment#47a3b841 key:string => long; // function
```

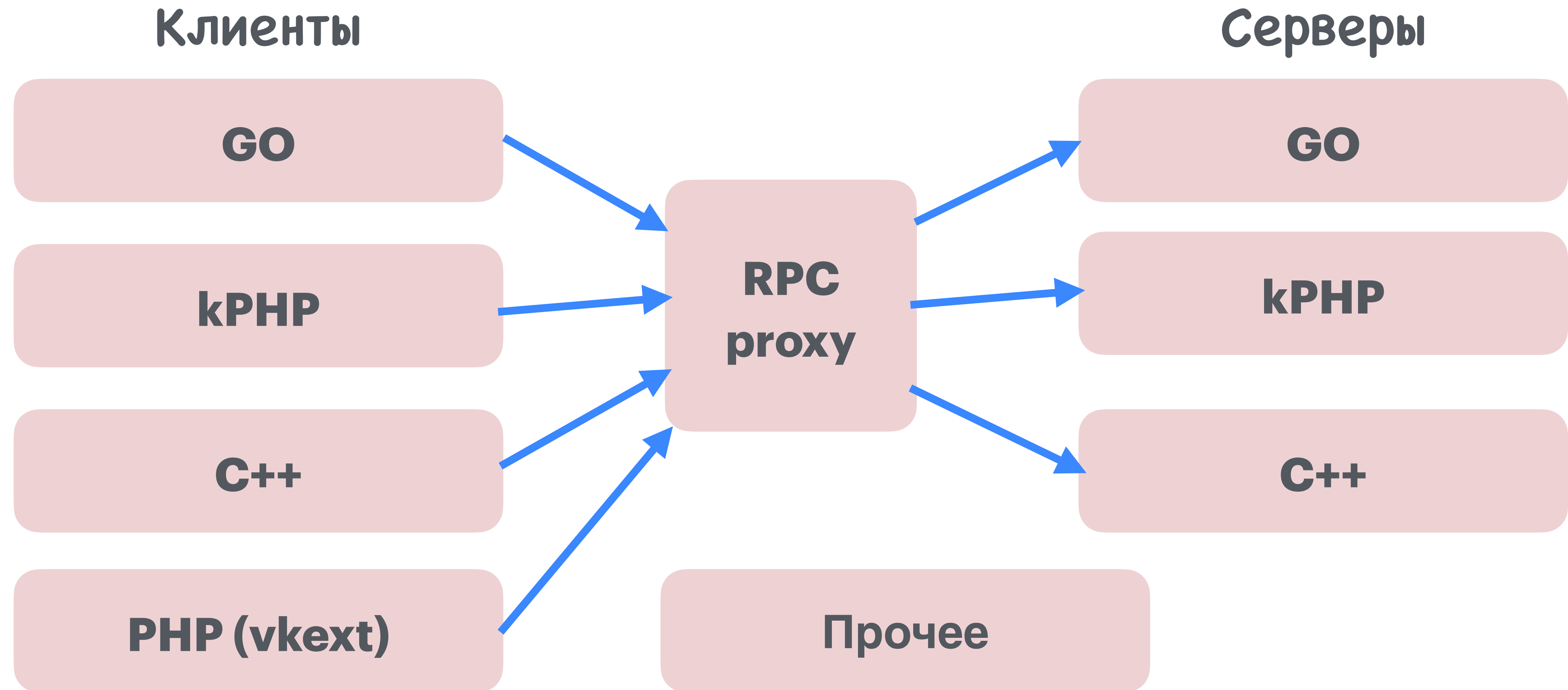
```
opt_bool = value?:int good:bool; // optional field & bool
```

```
pair <X, Y> = x:X y:Y; // template by type
```

```
hash <dim:uint> = x:[dim]byte; // template by unit & array
```

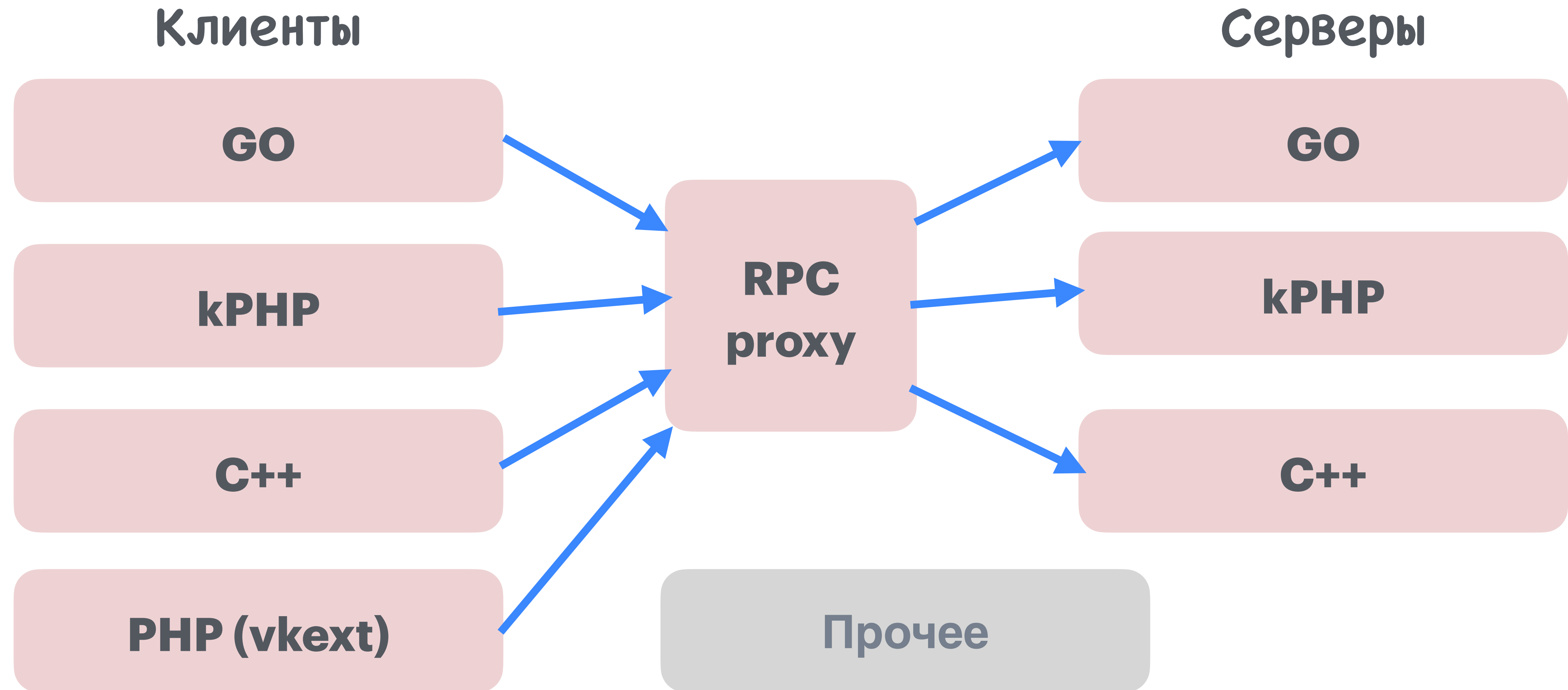

Что нужно перевести на TL2

В конечном счете все компоненты, пользующиеся RPC



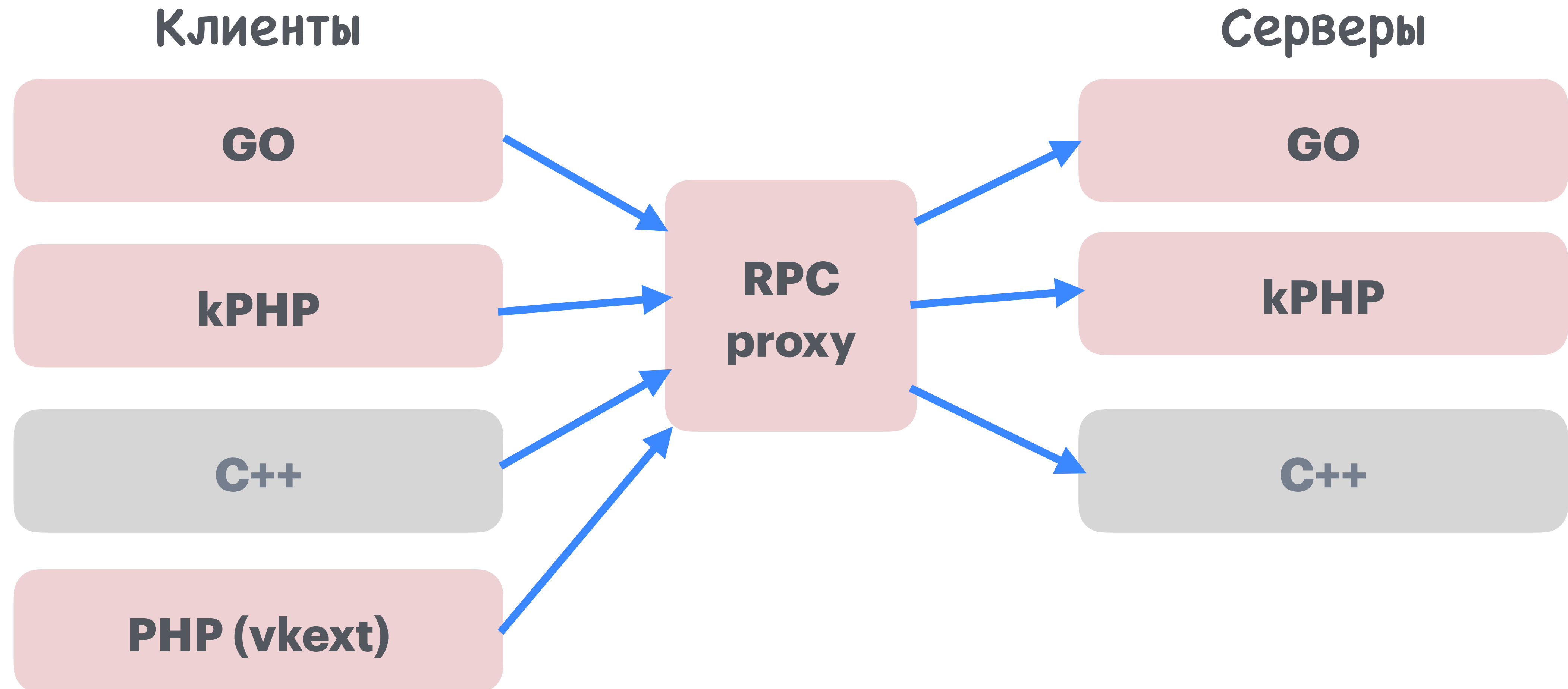
Что нужно перевести на TL2

Отложим прочее, так как мы мало про него знаем



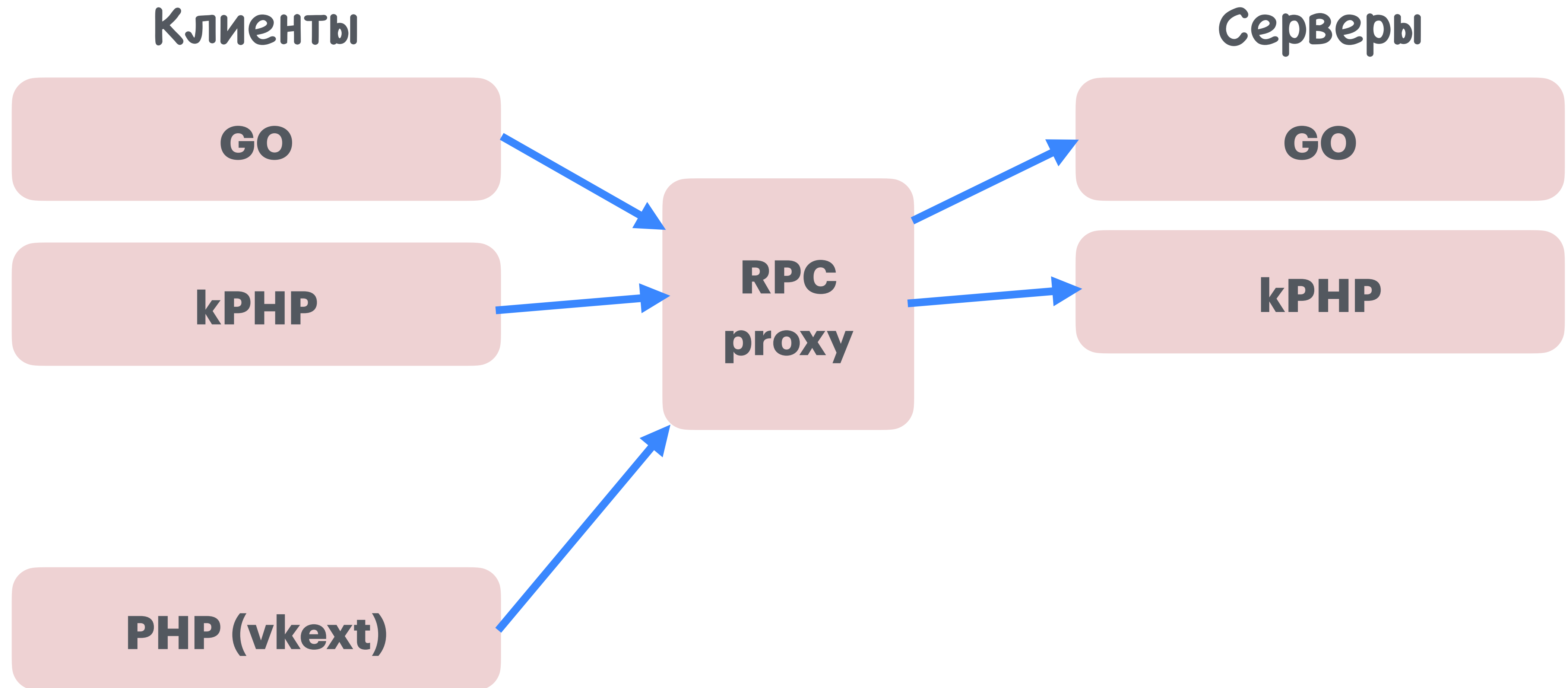
Что нужно перевести на TL2

Отложим движки, их API довольно стабильны



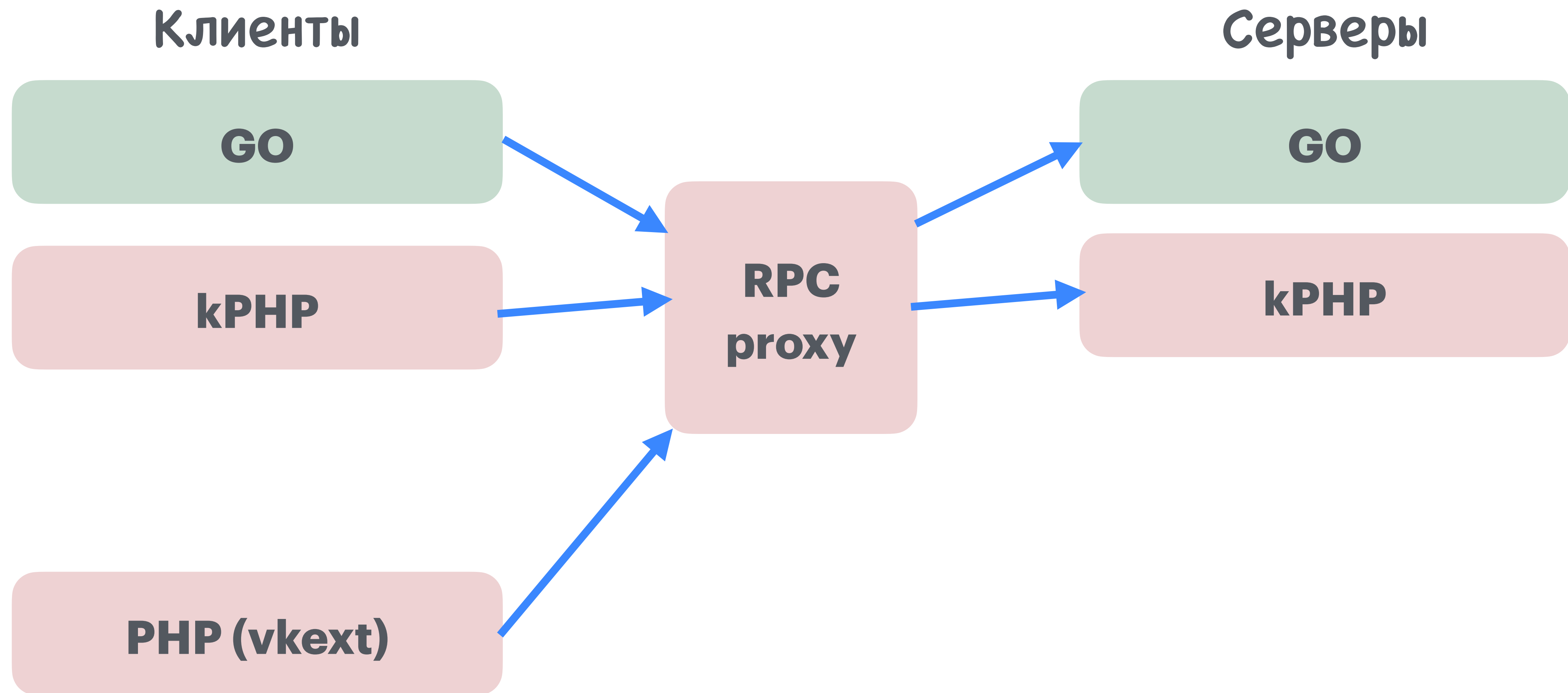
Что нужно перевести на TL2

Минимально полезный объём внедрения



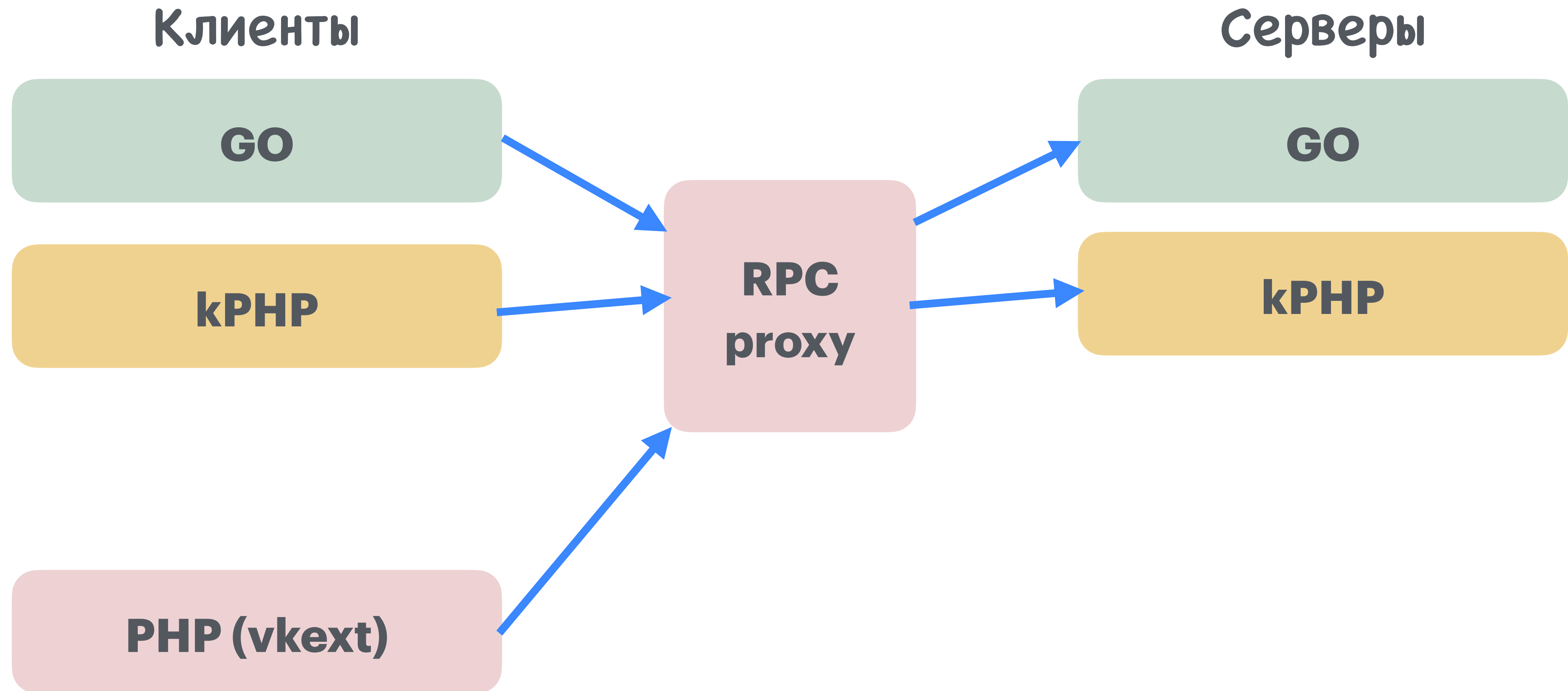
Что нужно перевести на TL2

На 99% готовы GO серверы и клиенты



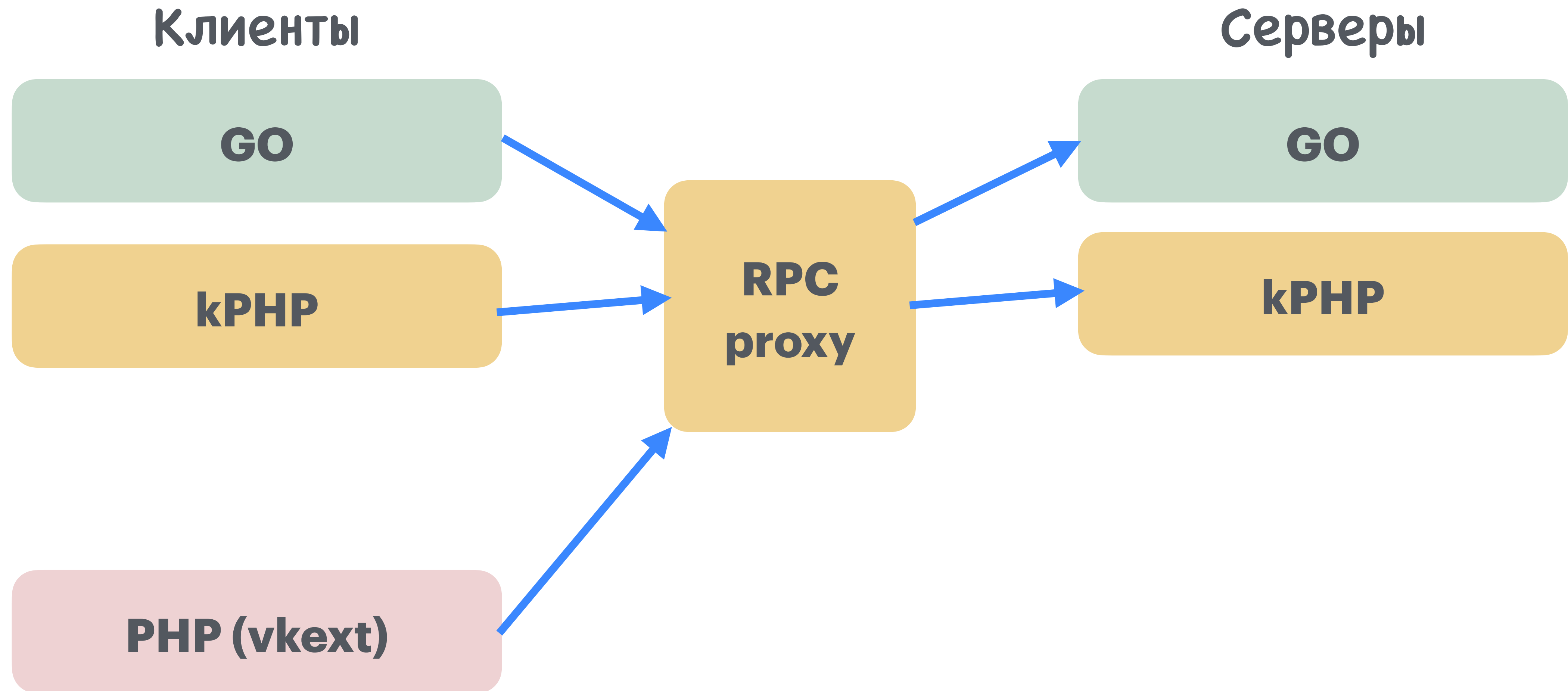
Что нужно перевести на TL2

Работающий прототип kPHP клиента и сервера



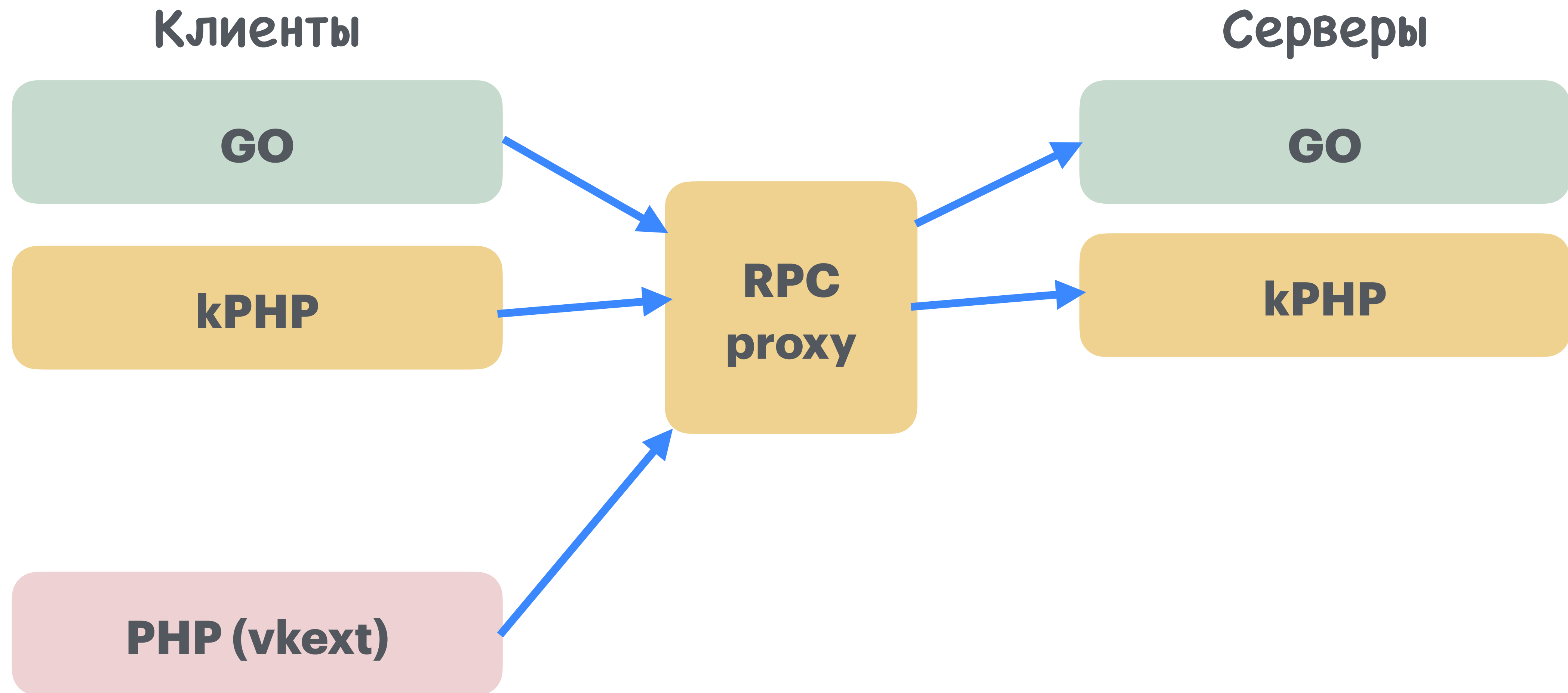
Что нужно перевести на TL2

Не ожидается проблем с RPC proxy



Что нужно перевести на TL2

Главный оставшийся риск это vkext



Изменения в формате RPC

Изменения в формате RPC

формат вызова

				67	dc	3a	22									
--	--	--	--	----	----	----	----	--	--	--	--	--	--	--	--	--

Extra

Magic

Body (TL1)

Изменения в формате RPC

формат вызова

				67	dc	3a	22										
--	--	--	--	----	----	----	----	--	--	--	--	--	--	--	--	--	--

Extra

Magic

Body (TL1)

Минимальные изменения

Изменения в формате RPC

формат вызова TL1 vs TL2

				67	dc	3a	22										
--	--	--	--	----	----	----	----	--	--	--	--	--	--	--	--	--	--

Extra

Magic

Body (TL1)

				29	32	4c	54	67	dc	3a	22						
--	--	--	--	----	----	----	----	----	----	----	----	--	--	--	--	--	--

Extra

TL2 Marker

Magic

Body (TL2)

Изменения в RPC Proxy

Изменения в RPC Proxy

Схемы шардирования

Изменения в RPC Proxy

Схемы шардирования – any alive (stateless сервисы)

				67	dc	3a	22										
--	--	--	--	----	----	----	----	--	--	--	--	--	--	--	--	--	--

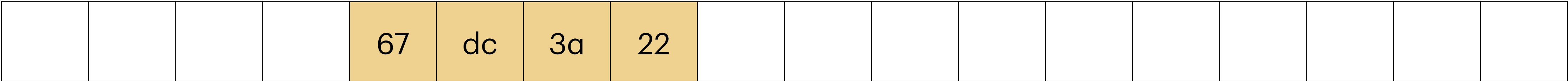
Extra

Magic

Body (TL1)

Изменения в RPC Proxy

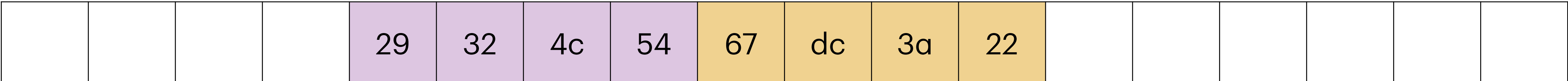
Схемы шардирования - any alive (stateless сервисы)



Extra

Magic

Body (TL1)



Extra

TL2 Marker

Magic

Body (TL2)

Изменения в RPC Proxy

Схемы шардирования – явное указание (int forward, etc.)

	05			67	dc	3a	22										
--	----	--	--	----	----	----	----	--	--	--	--	--	--	--	--	--	--

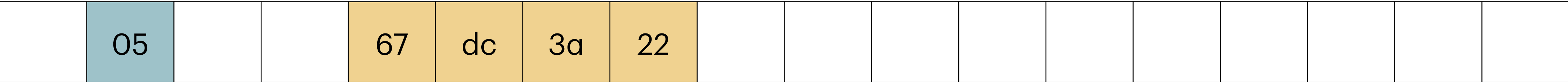
Extra

Magic

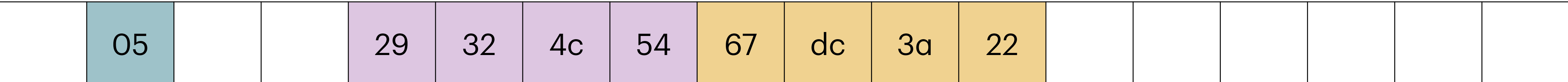
Body (TL1)

Изменения в RPC Proxy

Схемы шардирования – явное указание (int forward, etc.)



Extra		Magic		Body (TL1)
-------	--	-------	--	------------



Extra		TL2 Marker		Magic		Body (TL2)
-------	--	------------	--	-------	--	------------

Изменения в RPC Proxy

Схемы шардирования – first int

				67	dc	3a	22	05	00	00	00						
--	--	--	--	----	----	----	----	----	----	----	----	--	--	--	--	--	--

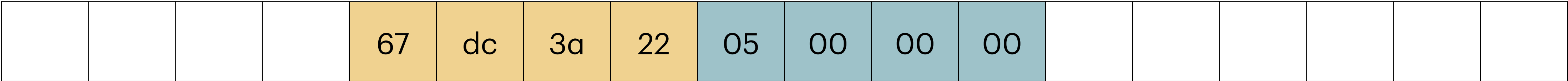
Extra

Magic

Body (TL1)

Изменения в RPC Proxy

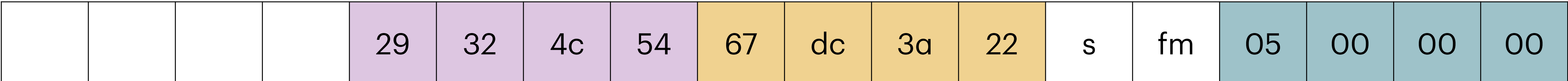
Схемы шардирования – first int



Extra

Magic

Body (TL1)



Extra

TL2 Marker

Magic

Body (TL2)

Изменения в RPC Proxy

Схемы шардирования

- any alive
- explicit (extra)
- first int, first string
- memcache (key prefix)
- lists, lists2 (сложная логика)
- etc

Изменения в RPC Proxy

Схемы шардирования

- any alive
- explicit (extra)
- first int, first string
- memcache (key prefix)
- lists, lists2 (сложная логика)
- etc

Дописываем по мере необходимости

Изменения в RPC Proxy

Схемы шардирования

- any alive
- explicit (extra)
- first int, first string
- memcache (key prefix)
- lists, lists2 (сложная логика)
- etc

TL1/TL2
Тэг в метриках

Дописываем по мере необходимости

Изменения в GO (server)

ИЗМЕНЕНИЯ В GO (server)

99% пользуются сгенерированным кодом

```
1 // inside service main
2     router := tlbug.Handler{
3         Create: func(args tlbug.Create) tlbug.CreateResponse {
4             ret := tlbug.CreateResponse{}
5             ... // fill response object
6             return ret
7         },
8         ...
9     };
10
11     serverRPC := rpc.NewServer(
12         rpc.ServerWithHandler(router.Handle),
13         ...
```


Изменения в GO (server)

99% пользуются сгенерированным кодом

```
1 // package tlbug; (generated code)
2 func (h *Handler) Handle(hctx *rpc.HandlerContext) (err error) {
3     switch hctx.RequestTag() {
4     case 0x9dcf7a68: // bug.create
5         var args Create
6         args.Read(hctx.Request)
7         ret := h.Create(ctx, args)
8         hctx.Response = args.WriteResult(hctx.Response, ret)
9     ...
```


Изменения в GO (server)

99% пользуются сгенерированным кодом

```
1 // package tlbug; (generated code)
2 func (h *Handler) Handle(hctx *rpc.HandlerContext) (err error) {
3     switch hctx.RequestTag() {
4     case 0x9dcf7a68: // bug.create
5         var args Create
6         if hctx.BodyFormatTL2() {
7             args.ReadTL2(hctx.Request)
8             ret := h.Create(ctx, args)
9             hctx.Response = args.WriteResultTL2(hctx.Response, ret)
10        } else {
11            args.Read(hctx.Request)
12            ret := h.Create(ctx, args)
13            hctx.Response = args.WriteResult(hctx.Response, ret)
14        }
15    ...
}
```

Изменения в GO (server)

Для поддержки TL2 нужно просто пересобрать и выкатить

Изменения в GO (client)

Изменения в GO (client)

99% пользуются сгенерированным кодом

```
1 // inside some service
2 func CreateBug(client *tlbug.Client) {
3     args := tlbug.Create{
4         Domain:    "global",
5         ShortName:  "bad",
6     }
7     ret := tlbug.CreateResponse{}
8     client.Create(args, nil, &ret)
9     ...
}
```

Изменения в GO (client)

99% пользуются сгенерированным кодом

```
1 // package tlbug; (generated code)
2 func (c *Client) Create(args Create, ret *CreateResponse) {
3     ...
4     req.Body = args.Write(req.Body)
5     resp, err := c.Client.Do(ctx, c.Network, c.Address, req)
6     args.ReadResult(resp.Body, ret)
7     ...
}
```


Изменения в GO (client)

99% пользуются сгенерированным кодом

```
1 // package tlbug; (generated code)
2 func (c *Client) Create(args Create, ret *CreateResponse) {
3     ...
4     req.BodyFormatTL2 = true
5     req.Body = args.WriteTL2(req.Body)
6     resp, err := c.Client.Do(ctx, c.Network, c.Address, req)
7     args.ReadResultTL2(resp.Body, ret)
8     ...
}
```

Изменения в GO (client)

99% пользуются сгенерированным кодом

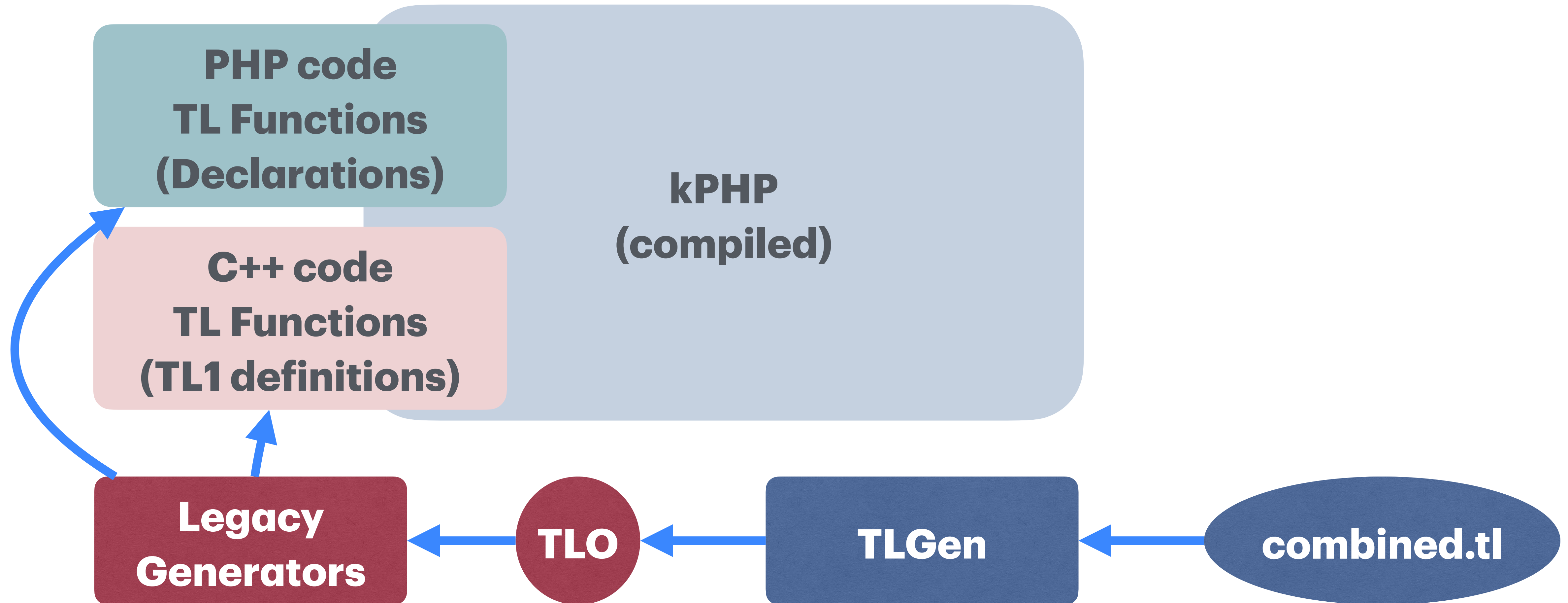
```
1 // package tlbug; (generated code)
2 func (c *Client) Create(args Create, ret *CreateResponse) {
3     ...
4     req.BodyFormatTL2 = true
5     req.Body = args.WriteTL2(req.Body)
6     resp, err := c.Client.Do(ctx, c.Network, c.Address, req)
7     args.ReadResultTL2(resp.Body, ret)
8     ...
}
```

Включается через whitelist после того, как все серверы обновлены

Изменения в (к)РНР

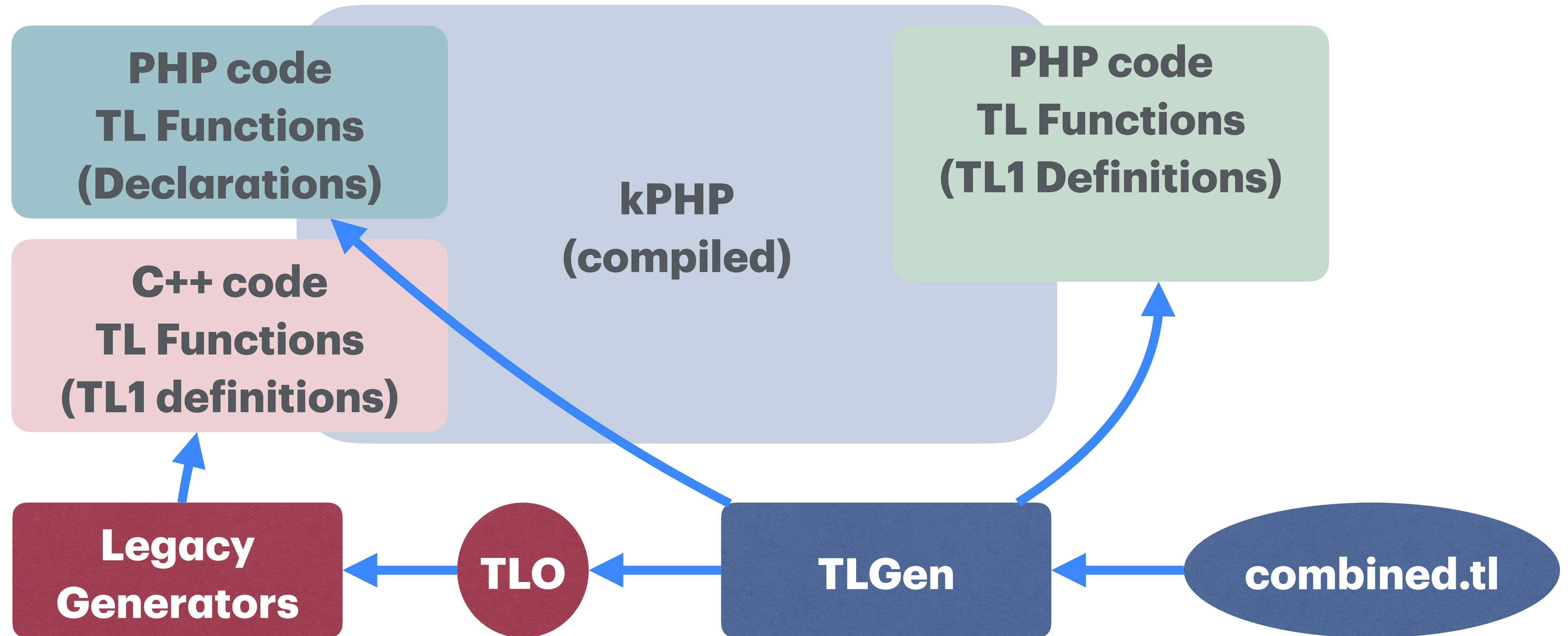
Изменения в кРНР

Сериализаторы TL в кРНР – сейчас



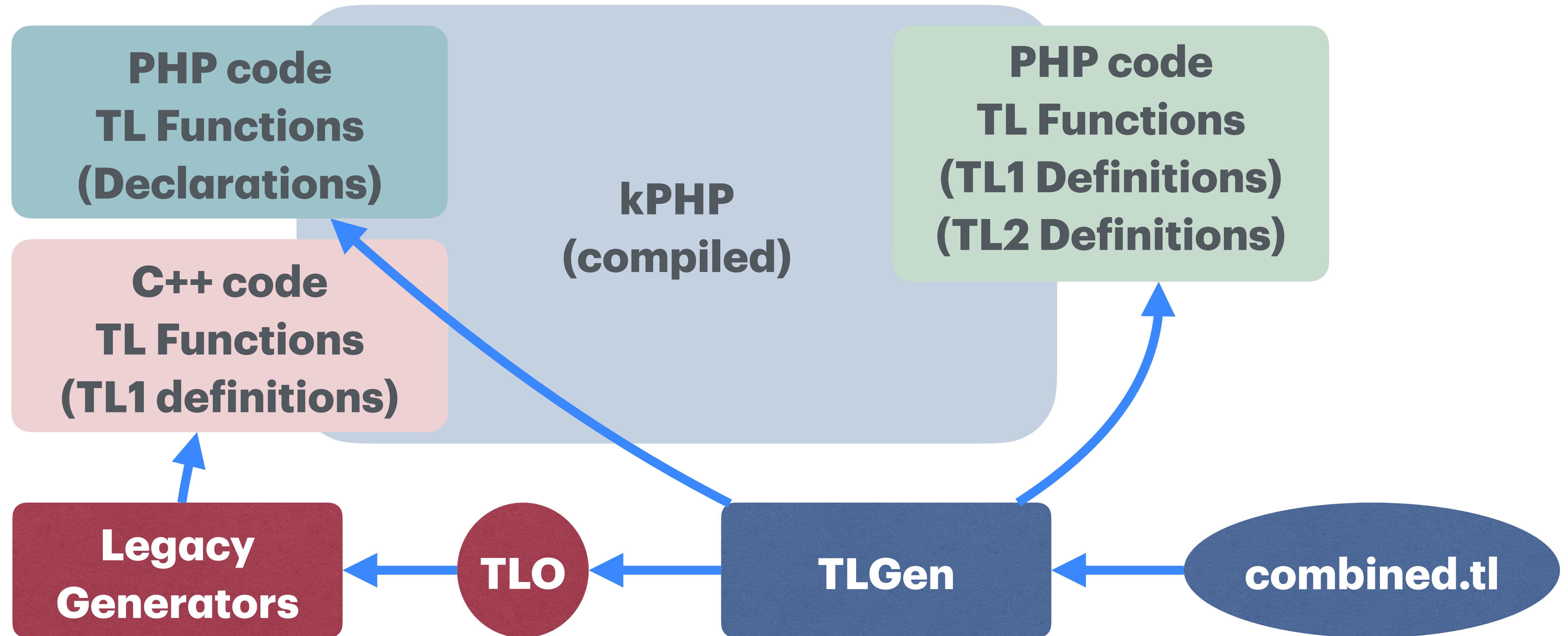
Изменения в кРНР

Сериализаторы TL в кРНР – переходный период



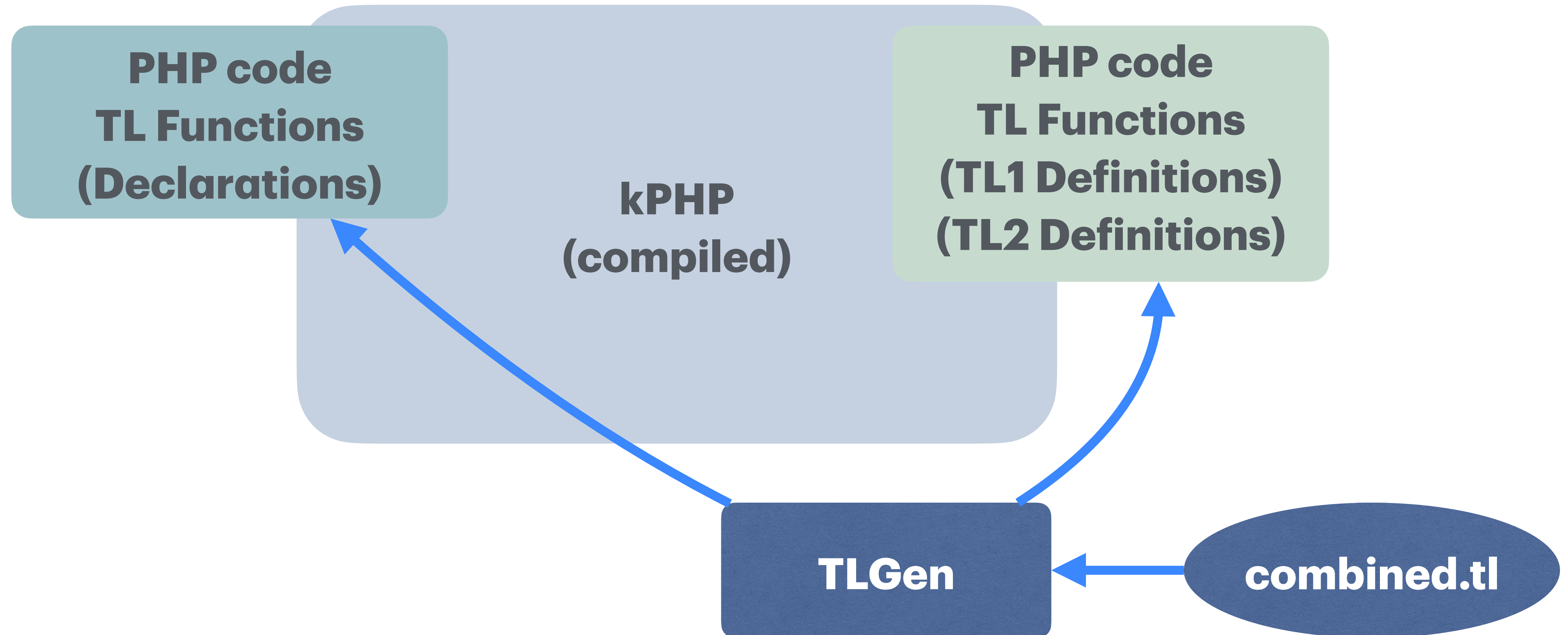
Изменения в кРНР

Сериализаторы TL в кРНР – переходный период



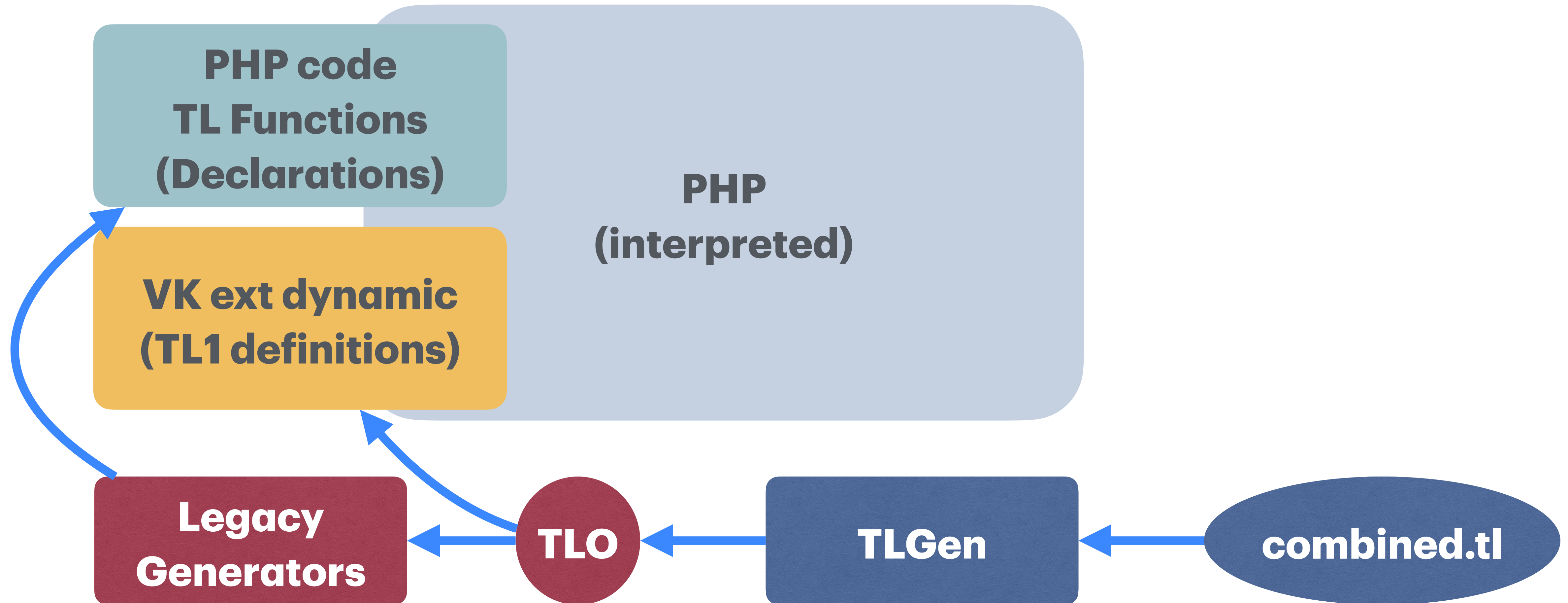
Изменения в kPHP

Сериализаторы TL в kPHP – финал



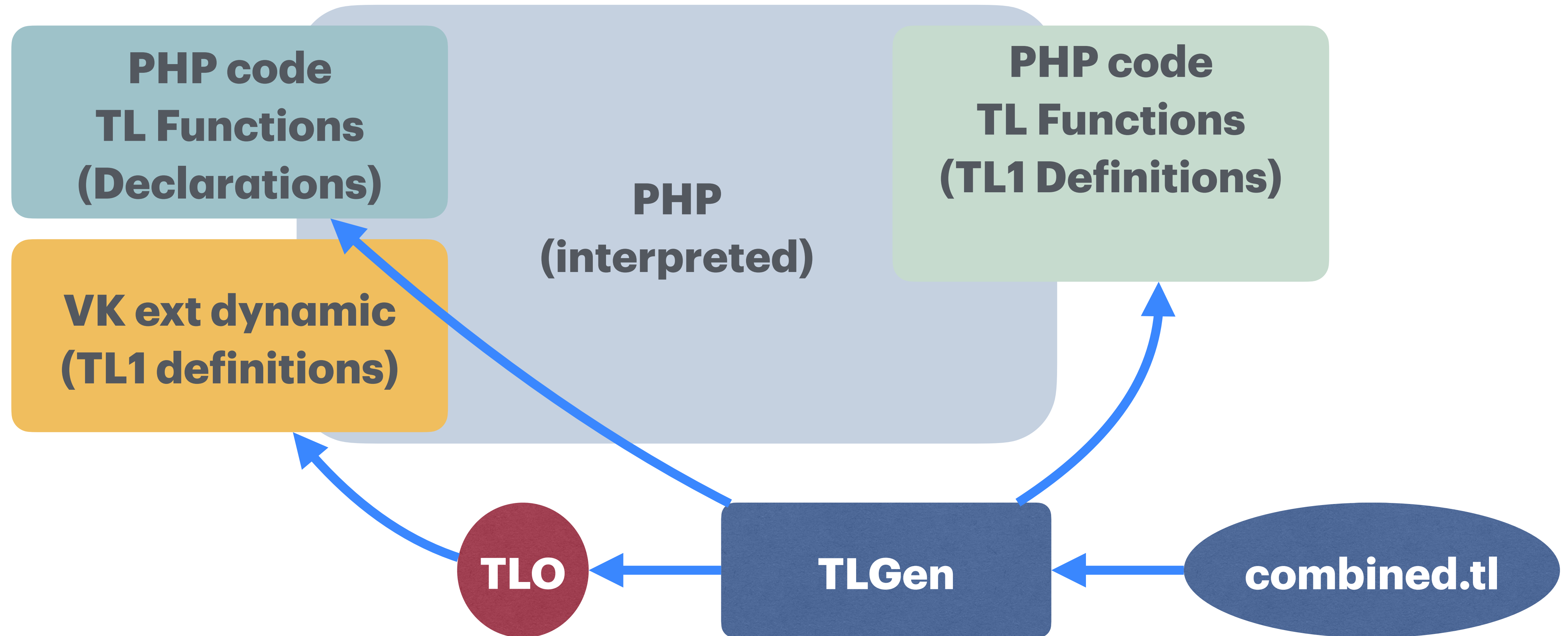
Изменения в PHP (vkext)

Сериализаторы TL в PHP – сейчас



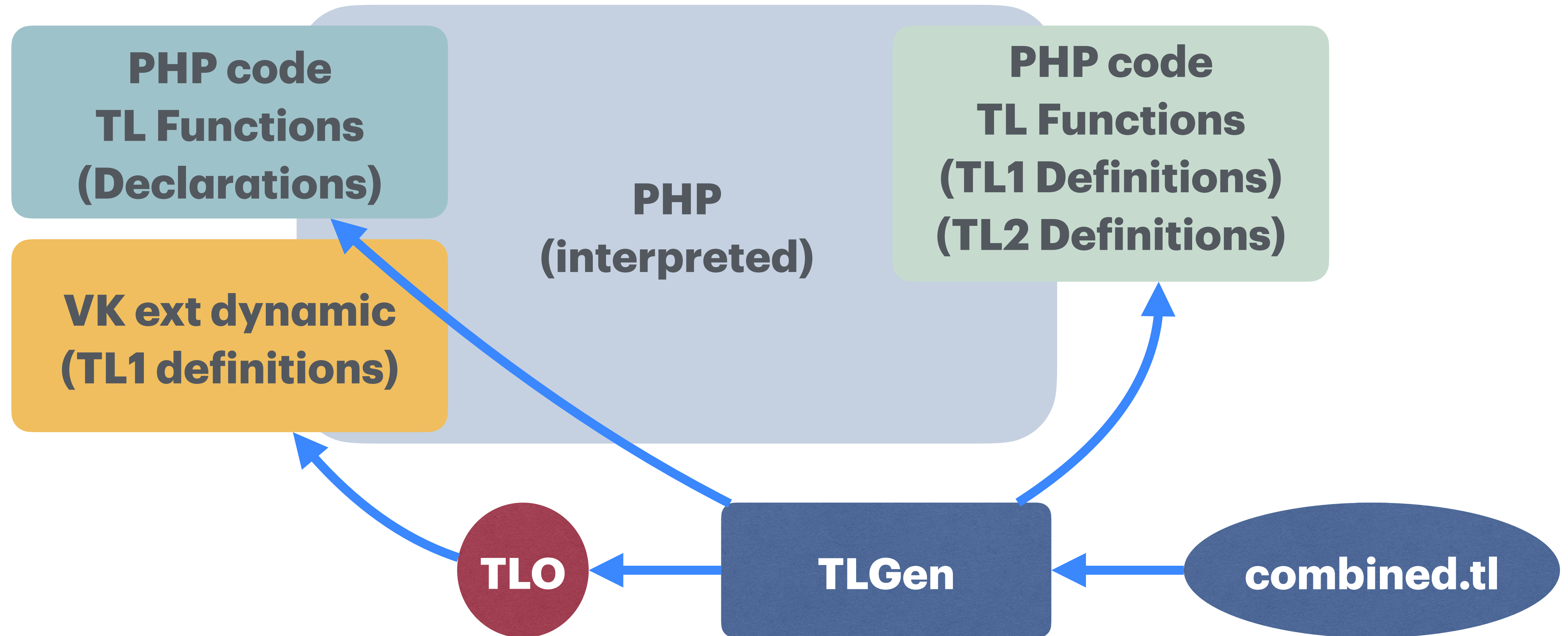
Изменения в PHP (vkext)

Сериализаторы TL в PHP – переходный период



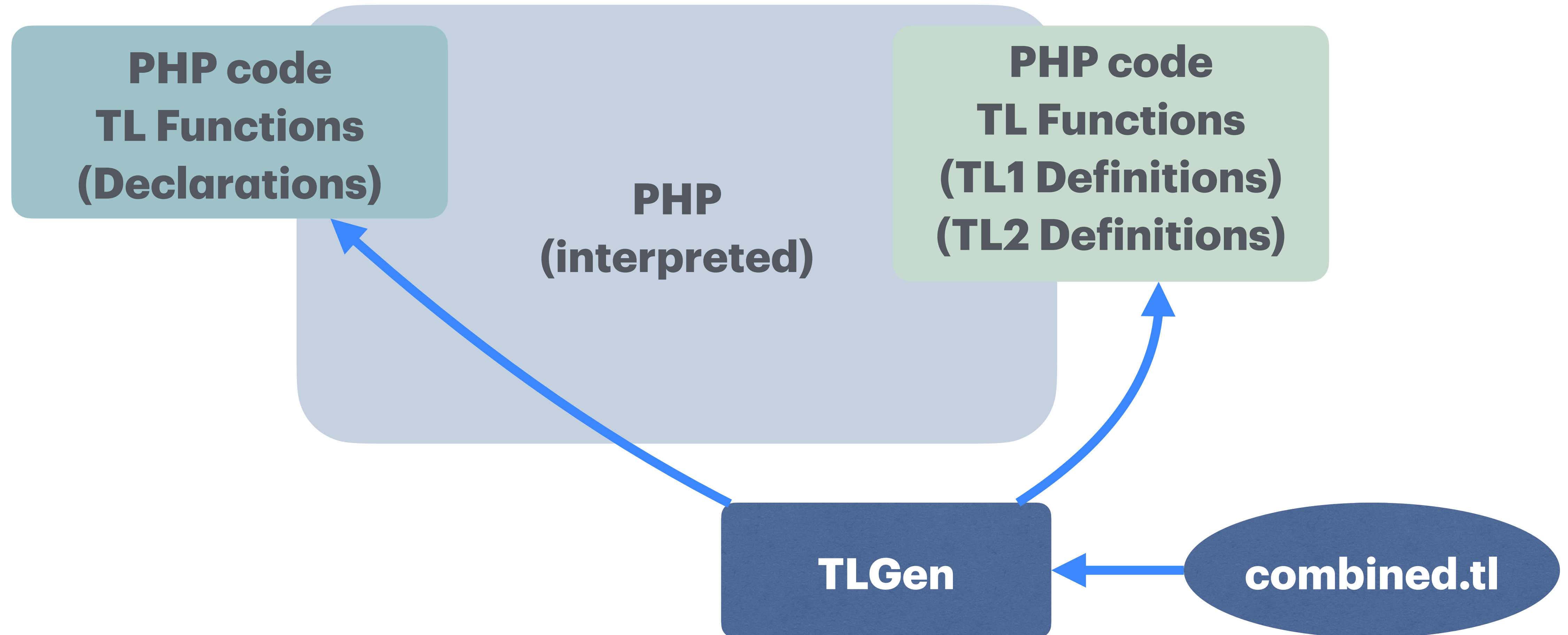
Изменения в PHP (vkext)

Сериализаторы TL в PHP – переходный период



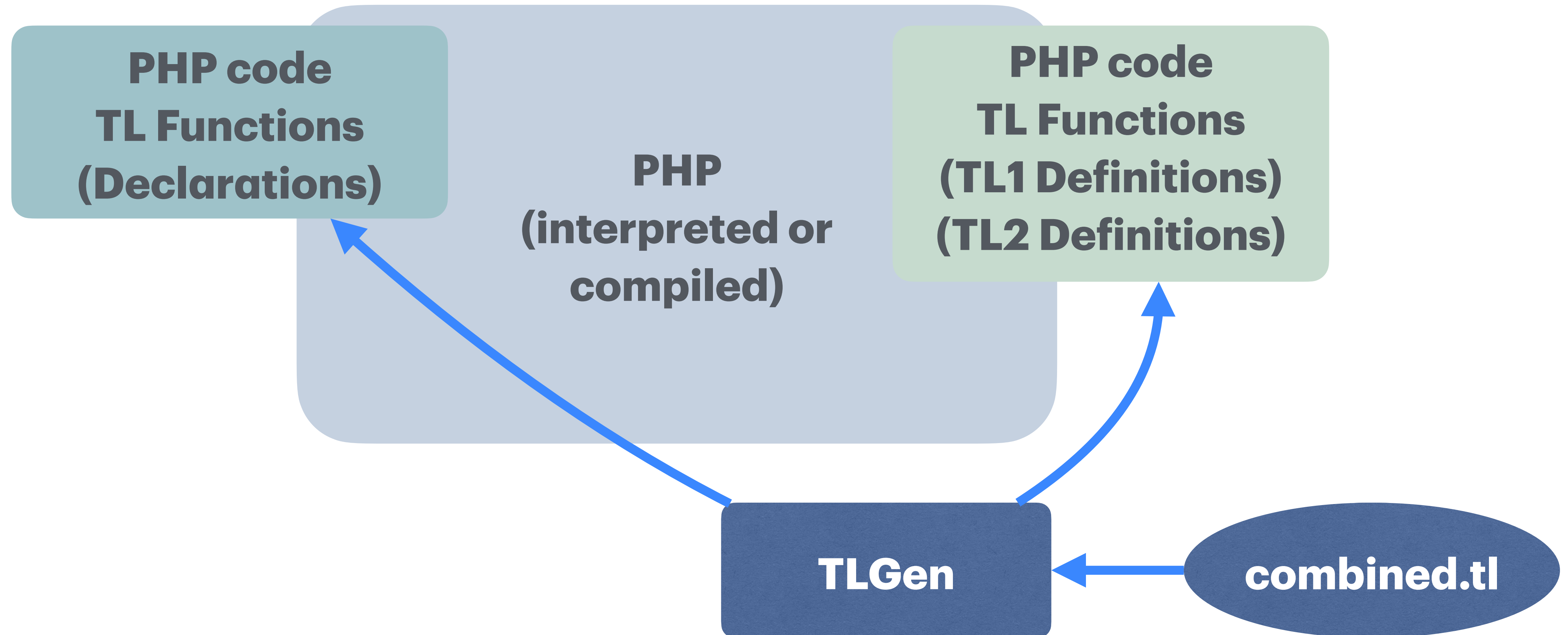
Изменения в PHP (vkext)

Сериализаторы TL в PHP – финал



Изменения в (к)PHP

Сериализаторы TL в (к)PHP – финал



Изменения в (к)PHP

Пользовательский код не меняется

```
1 use VK\TL\bug\Functions;  
2 ...  
3 $query = new Functions\bug_create("total", "bad");  
4 $rpc_response = $client->callSynchronously($query);  
5 $result = Functions\bug_create::result($response);  
6 ...
```

Это справедливо и для различных обёрток

Изменения в (к)PHP

Базовый класс VK\TL\RPCFunction

```
1 namespace VK\TL;  
2 /**  
3  * @kphp-tl-class  
4  */  
5 interface RpcFunction {  
6     public function getTLFunctionName():string;  
7 }
```


Изменения в (к)PHP

Делается точка кастомизации

```
1 namespace VK\TL;
2 /**
3  * @kphp-tl-class
4  */
5 interface RpcFunction {
6     public function getTLFunctionName():string;
7     /**
8      * @return \RpcFunctionFetcher
9      */
10    public function custom_fetcher();
11 }
```

Изменения в (к)PHP

Изначально из всех TL функций возвращаем null

```
1 class bug_create implements TL\RpcFunction {  
2     public function custom_fetcher(): \RpcFunctionFetcher {  
3         return null;  
4     }  
5 ...
```


Изменения в (к)PHP

Новый код добавляется по whitelist в генераторе, сохраняет запрос

```
1 class bug_create implements TL\RpcFunction {
2     public function custom_fetcher(): \RpcFunctionFetcher {
3         set_last_stored_tl_function_magic(0xd33b13ae);
4         store_int(0xd33b13ae);
5         $this->write_tl1();
6         return new bug_create_fetcher($this->fm);
7     }
8     private function write_tl1() {
9         store_string($this->name);
10        store_string($this->domain);
11    }
12 ...
```


Изменения в (к)PHP

Также генерируется `fetcher`, который читает ответ

```
1 class bug_create_fetcher implements \RpcFunctionFetcher {
2     private int $fm = 0;
3
4     public function typed_fetch(): \VK\TL\RpcFunctionReturnResult {
5         $ret = new bug_create_result();
6         $ret->$id = fetch_int();
7         if ($this->fm & 0x10000) {
8             $ret->$url = fetch_string();
9         }
10        return $ret;
11    }
12 ...
```


Изменения в (к)РНР

Новое поведение включается ручкой для каждого namespace

```
1 class bug_create implements TL\RpcFunction {
2     public function custom_fetcher():\RpcFunctionFetcher {
3         switch(new_tl_serializers_enabled("bug") {
4             case 1:
5                 ...
6                 return new bug_create_fetcher($this->fm);
7             case 2:
8                 ...
9                 return new bug_create_fetcher_tl2();
10        }
11        return null;
12    }
13 ...
```

Изменения в (k)RPC

Новый код поддерживает только типизированный RPC

Нам нужно выпилить весь нетипизированный RPC и LongID

Взаимосвязь TL1 и TL2

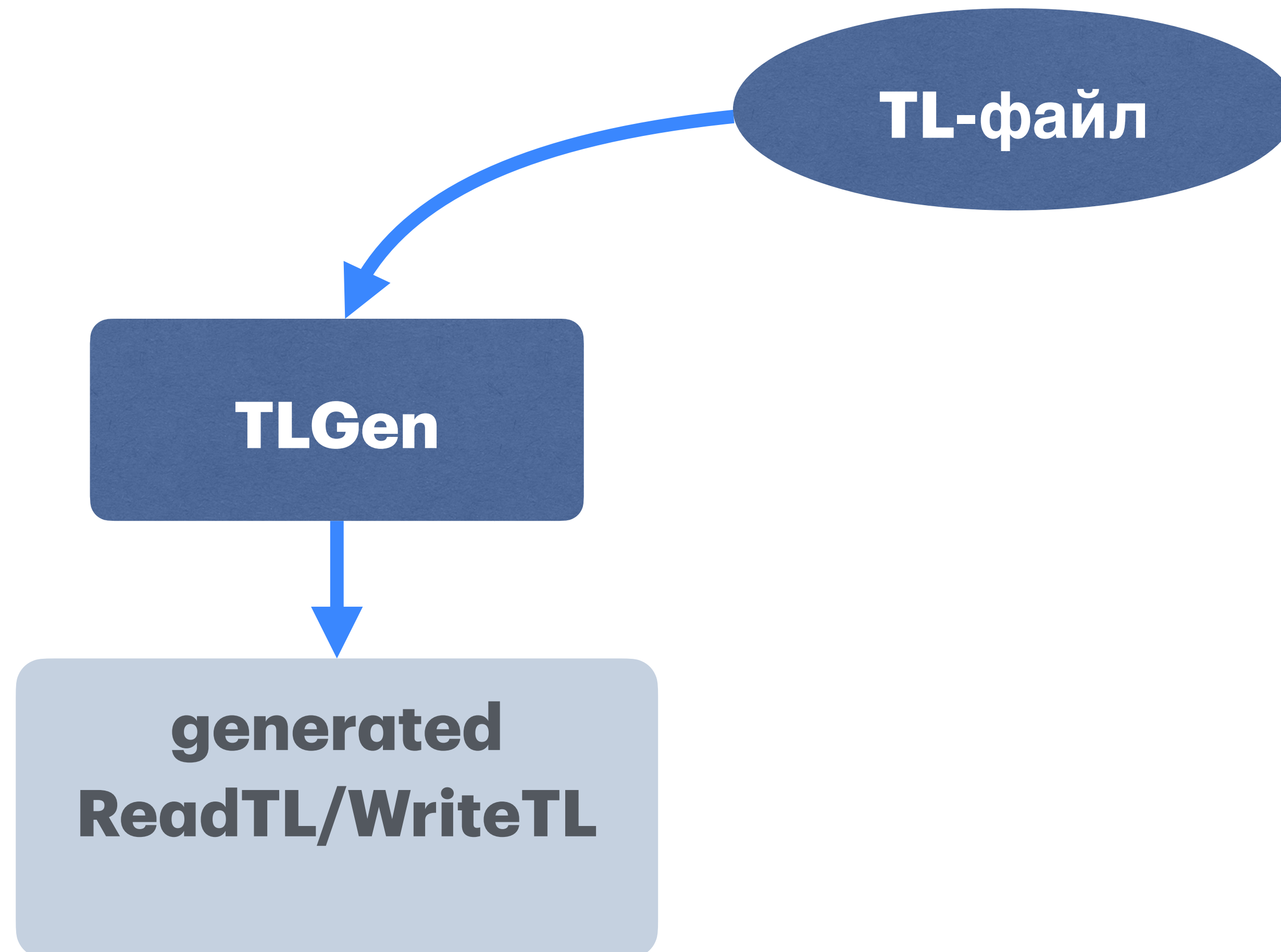
Взаимосвязь TL1 и TL2

Откуда у одного и того же типа методы и TL1, и TL2?

```
1 // package tlbug; (generated code)
2 func (h *Handler) Handle(hctx *rpc.HandlerContext) (err error) {
3     switch hctx.RequestTag() {
4     case 0x9dcf7a68: // bug.create
5         → var args Create
6         if hctx.BodyFormatTL2() {
7             → args.ReadTL2(hctx.Request)
8             ret := h.Create(ctx, args)
9             hctx.Response = args.WriteResultTL2(hctx.Response, ret)
10        } else {
11            → args.Read(hctx.Request)
12            ret := h.Create(ctx, args)
13            hctx.Response = args.WriteResult(hctx.Response, ret)
14        }
15    ...
}
```

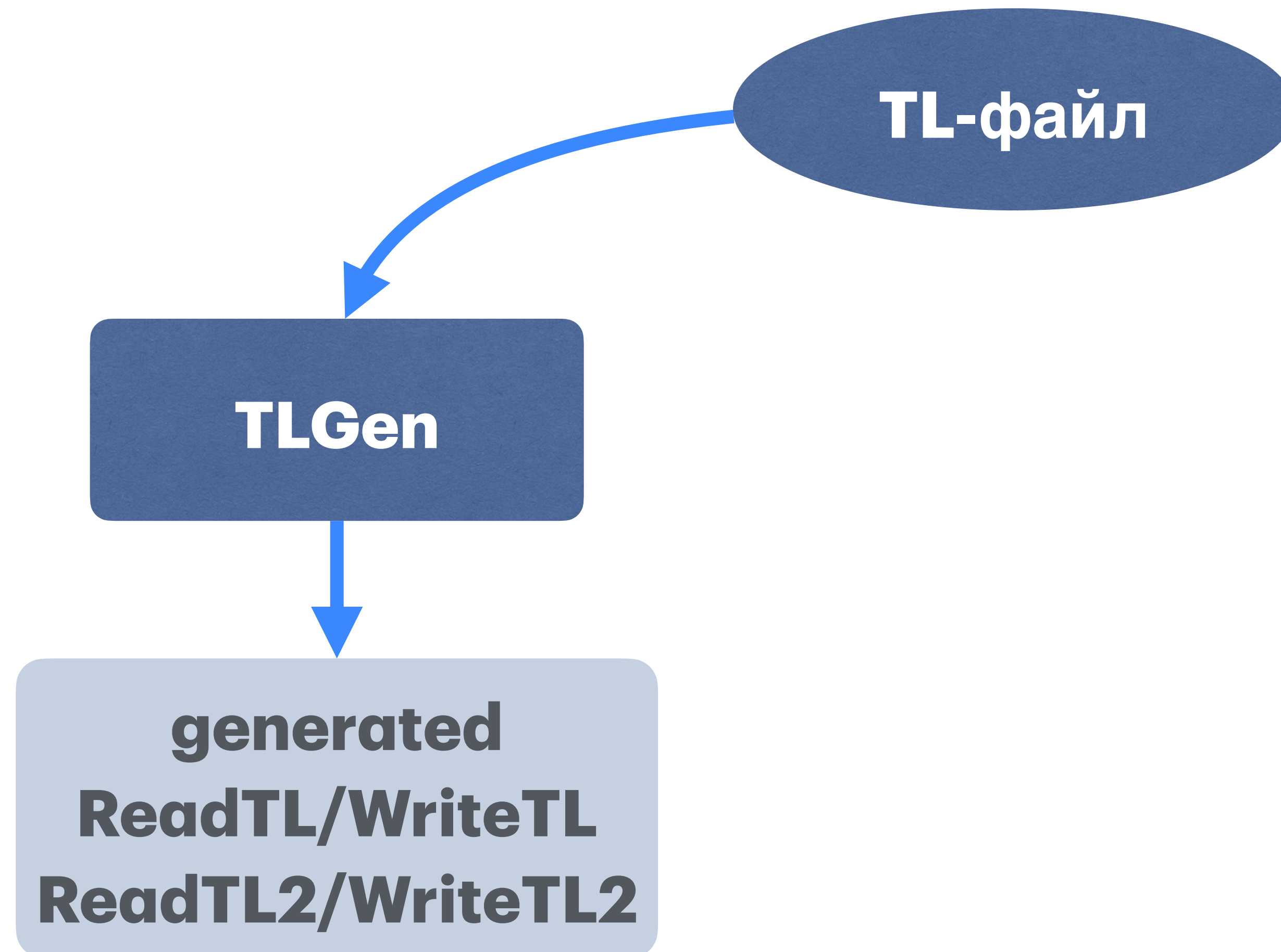

Взаимосвязь TL1 и TL2

Генерация TL1 сериализаторов



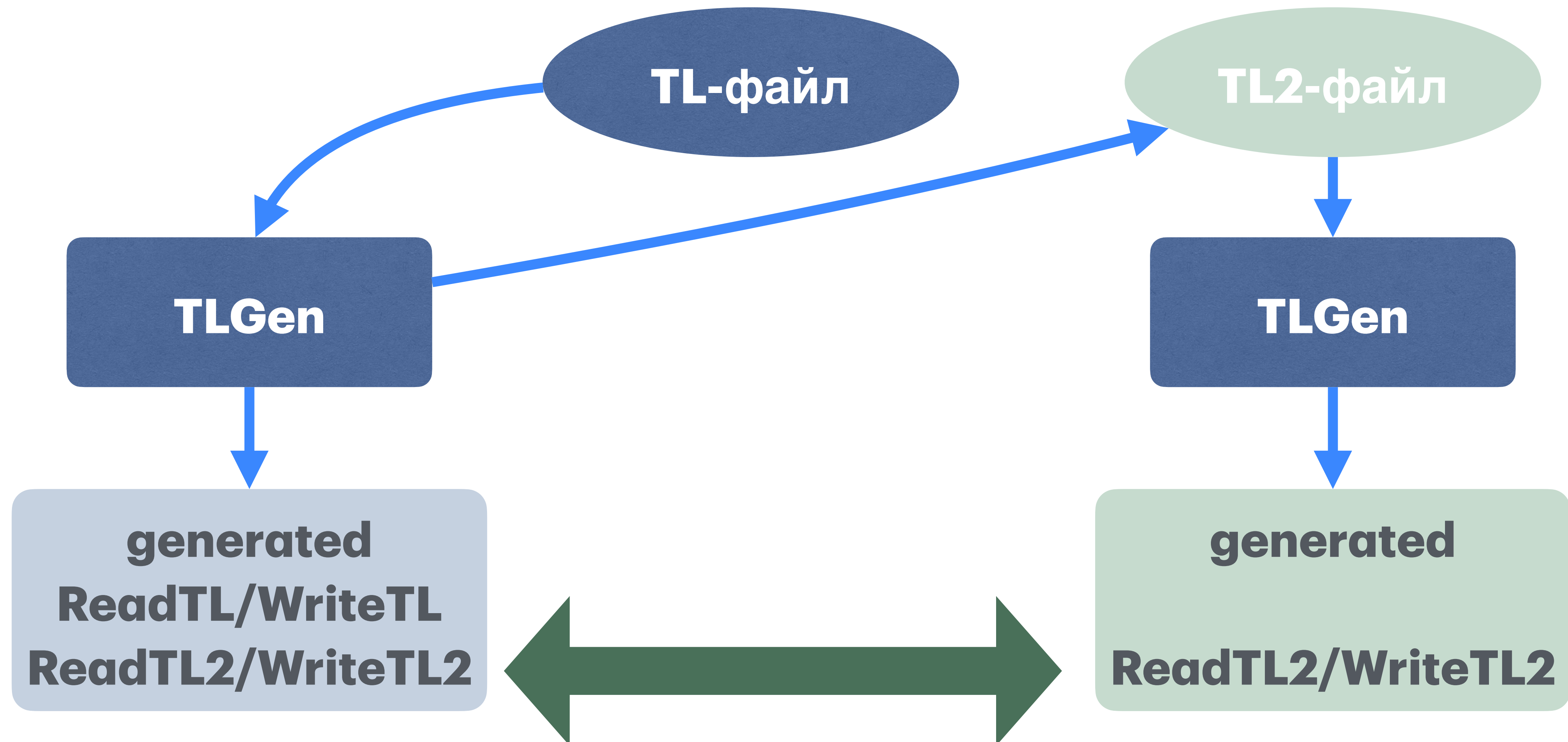
Взаимосвязь TL1 и TL2

Генерация TL2 сериализаторов



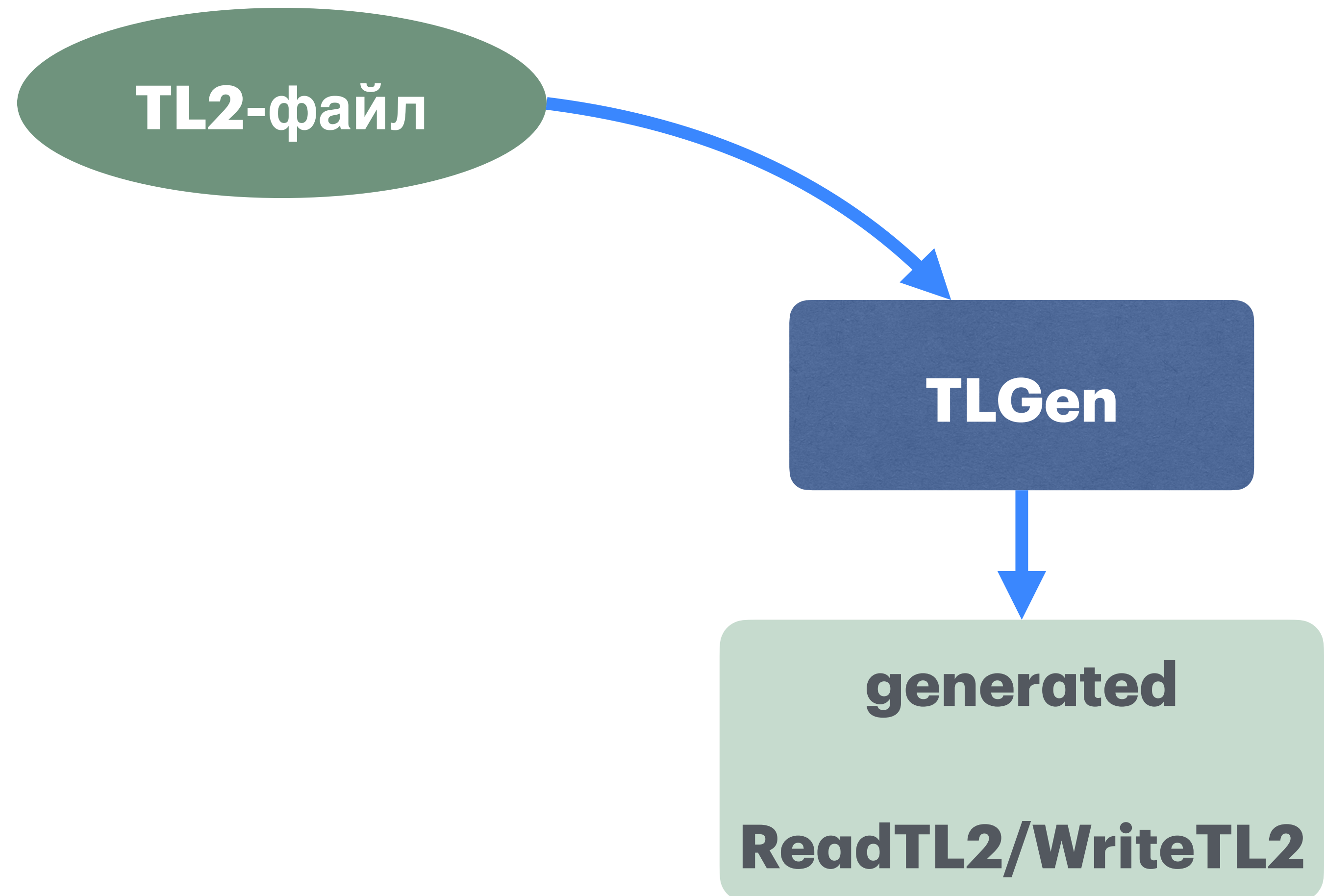
Взаимосвязь TL1 и TL2

Генерация TL2 сериализаторов и TL2 файла



Взаимосвязь TL1 и TL2

замена TL файла на сгенерированный TL2 файл



Пример перехода на TL2

Пример перехода на TL2

Гранулярность перехода – TL namespace

Пример перехода на TL2

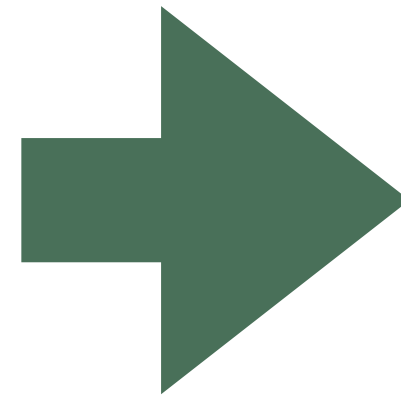
Исходный TL-файл

```
1 // tl/tl/bug.tl
2 bug.createResponse#1b084952 {fm:#}
3   id   : int
4   url  : fm.16?string
5   = bug.CreateResponse fm;
6
7 ---functions---
8
9 @readwrite bug.create#9dcf7a68
10  fm      : #
11  name    : string
12  domain  : fm.0?string
13  return_url : fm.16?true
14  = (bug.CreateResponse fm);
```


Пример перехода на TL2

Сгенерированный TL2-файл

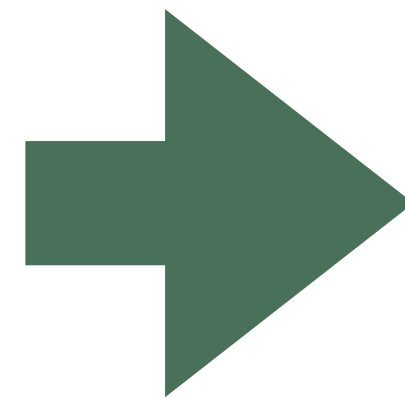
```
1 // tl/tl/bug.tl
2 bug.createResponse#1b084952 {fm:#}
3   id : int
4   url : fm.16?string
5   = bug.CreateResponse fm;
6
7 ---functions---
8
9 @readwrite bug.create#9dcf7a68
10  fm      : #
11  name     : string
12  domain   : fm.0?string
13  return_url : fm.16?true
14  = (bug.CreateResponse fm);
```



```
1 // tl/tl/bug.tl2
2 bug.createResponse#1b084952 =
3   id : int
4   url?: string;
5
6
7
8
9 @readwrite bug.create#9dcf7a68
10  fm      : #
11  name     : string
12  domain?  : string
13  return_url : bool
14  => bug.createResponse;
```


Пример перехода на TL2

Дальнейшая ручная трансформация TL2-файла



```
1 // tl/tl/bug.tl2
2 bug.createResponse#1b084952 =
3   id   : int
4   url?: string;
5
6
7
8
9 @readwrite bug.create#9dcf7a68
10  fm      : #
11  name     : string
12  domain?  : string
13  return_url : bool
14  => bug.createResponse;
```

```
1 // tl/tl/bug.tl2
2 bug.createResponse =
3   id   : int
4   url  : string;
5
6
7
8
9 @readwrite bug.create#9dcf7a68
10  _      : #
11  name    : string
12  domain  : string
13  _       : bool
14  => bug.createResponse;
```

Пример перехода на TL2

Последовательность действий при переходе сервиса на Go

- В репе vkgo добавляем namespace в список TL2 серверов
- Пересобираем серверы, дожидаясь раскатки
- В vkgo, vkcom добавляем namespace в список TL2 клиентов
- Ждем раскатки всех клиентов Go и PHP
- Заменяем TL файл в репе tl/tl на сгенерированный TL2 файл
- Опционально: убираем маски полей, меняем опциональность

Пример перехода на TL2

Последовательность действий при переходе сервиса на Go

- В репе vkgo добавляем namespace в список TL2 серверов
- Пересобираем серверы, дожидаясь раскатки
- В vkgo, vkcom добавляем namespace в список TL2 клиентов
- Ждем раскатки всех клиентов Go и PHP
- Заменяем TL файл в репе tl/tl на сгенерированный TL2 файл
- Опционально: убираем маски полей, меняем опциональность

Нам нужно выпилить весь нетипизированный RPC и LongID

Вопросы?

Григорий Бутейко [vk:@hrissan](https://vk.com/hrissan)